

Instrukcja do ćwiczeń laboratoryjnych

Nazwa przedmiotu	Eksploracja danych
Numer ćwiczenia	2
Temat ćwiczenia	Klasyfikacja drzewami decyzyjnymi i lasami losowymi: przycinanie, interpretowalność, k-NN

Poziom studiów	II stopień
Kierunek	Data Science
Forma studiów	Stacjonarne
Semestr	2

Wojciech Czech

1. Cel ćwiczenia

- Praktyczne zapoznanie się z klasyfikacją za pomocą drzew decyzyjnych, klasyfikatorem k -NN oraz lasami losowymi.
- Dogłębne zrozumienie i praktyczna implementacja *strategii przycinania drzewa*: pre-pruning (ograniczenia na etapie budowy) oraz post-pruning (*Cost-Complexity Pruning*, Breiman).
- Opanowanie nowoczesnych narzędzi wizualizacji drzew (`graphviz`, `dtreeviz`) oraz wyjaśniania modeli (built-in importance, *permutation importance*, wartości SHAP).
- Zrozumienie teoretycznych podstaw lasów losowych: dekompozycji bias-variance, bagging, dekorelacji drzew przez losowy podzbiór cech, oraz estymatora *out-of-bag*.
- Zastosowanie nowoczesnych metod optymalizacji hiperparametrów (Bayesian optimization z Optuną).

2. Wprowadzenie do ćwiczenia

Drzewa decyzyjne to rodzina algorytmów uczenia maszynowego realizujących zadanie klasyfikacji (lub regresji) poprzez rekursywny podział przestrzeni cech hiperpłaszczyznami równoległymi do osi. Każdy taki podział wyznaczany jest przez minimalizację wybranego kryterium nieczystości (zanieczyszczenia Giniego lub entropii warunkowej). Drzewa są wysoko interpretowalne, niewymagające skalowania cech i dobrze radzą sobie z cechami mieszanymi. Podstawowym problemem pojedynczego drzewa jest *wysoka wariancja* estymatora: drobne zmiany danych treningowych prowadzą do radykalnie odmiennych struktur drzewa. W rozwiązaniu tego problemu stosuje się dwie komplementarne techniki: *przycinanie* (dla pojedynczego drzewa) oraz *agregację wielu drzew* (bagging, lasy losowe, gradient boosting).

Klasyfikator k -najbliższych-sąsiadów (k -NN) jest najprostszym modelem nieparametrycznym i służy jako sensowna baza porównawcza. Jego zachowanie istotnie zależy od wyboru metryki, skalowania cech oraz wymiarowości przestrzeni (*curse of dimensionality*).

3. Przykładowe dane

- **Adult Census Income** (<https://www.openml.org/d/1590>)
Zbiór ze spisu USA z 1994 roku, zawierający ok. 48 842 rekordów opisujących osoby pracujące. Zadaniem jest klasyfikacja binarna: czy dana osoba zarabia ponad 50 tys. USD rocznie. Dostępny bezpośrednio przez `sklearn.datasets.fetch_openml('adult', version=2)`. Cechy:
 - *age*, *fnlwgt*, *education-num*, *capital-gain*, *capital-loss*, *hours-per-week* (numeryczne)
 - *workclass*, *education*, *marital-status*, *occupation*, *relationship*, *race*, *sex*, *native-country* (kategoryczne)Zbiór jest niezrównoważony pod względem proporcji klas (ok. 24% pozytywnych), zawiera braki danych w cechach `workclass`, `occupation`,

- `native-country`, oraz jest klasycznym benchmarkiem dla dyskusji o *fairness* modeli uczenia maszynowego (płeć, rasa jako cechy chronione).
- **Palmer Penguins** (<https://www.openml.org/d/42585>)
Nowoczesny następca Irisa, 344 pingwiny z trzech gatunków (*Adelie*, *Chinstrap*, *Gentoo*) z archipelagu Palmer (Antarktyda).
Dostępny przez `fetch_openml('penguins')`.
- **Zbiór NYT z poprzedniego laboratorium** (<http://home.agh.edu.pl/~czech/vis-datasets/misc/nyt-frame.csv>)
(101×4433 , TF-IDF dla artykułów NYT w dwóch kategoriach).

4. Przydatne biblioteki

1. Pandas, NumPy: operacje na danych tabelarycznych.
2. `scikit-learn` ≥ 1.3 :
 - `ColumnTransformer`, `Pipeline`, `OrdinalEncoder`, `SimpleImputer`
 - `DecisionTreeClassifier` (w szczególności metody `cost_complexity_pruning_path()`, atrybut `ccp_alpha`)
 - `RandomForestClassifier`, `HistGradientBoostingClassifier`
 - `KNeighborsClassifier`
 - `GridSearchCV`, `RandomizedSearchCV`, `StratifiedKFold`, `cross_val_score`
 - `permutation_importance` (z `sklearn.inspection`)
3. `graphviz` (Python binding + systemowy pakiet `graphviz`).
4. `dtreeviz` <https://github.com/parrrt/dtreeviz>: nowoczesna wizualizacja drzew z histogramami węzłów.
5. `shap`: <https://shap.readthedocs.io>.
6. `optuna` <https://optuna.org>: nowoczesny framework optymalizacji hiperparametrów (alternatywa: `scikit-optimize`).
7. Seaborn, Matplotlib.

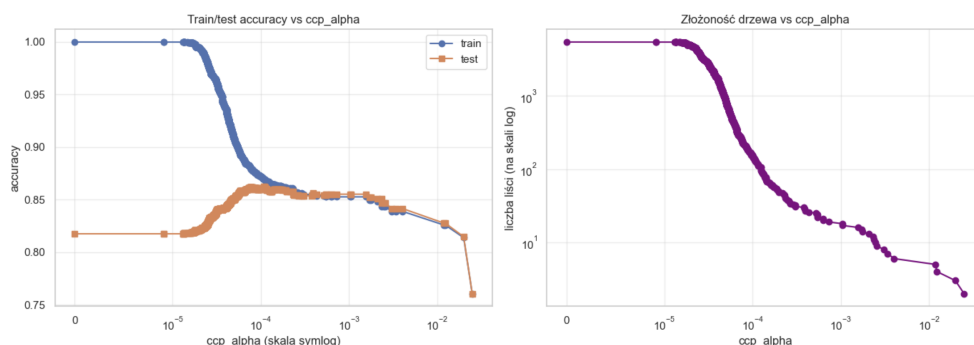
5. Przetwarzanie wstępne dla modeli drzewiastych

Drzewa decyzyjne są *niewrażliwe na monotoniczne transformacje cech*: każdy podział używa wyłącznie uporządkowania wartości. W szczególności:

- **Nie stosujemy `StandardScaler` przed drzewem**: skalowanie nie zmienia struktury drzewa, a utrudnia interpretację progów podziału.
- Cechy katagoryczne nominalne można zakodować przez `OrdinalEncoder`: dla drzew porządek narzucony przez kodowanie nie wprowadza problemów (algorytm może rozdzielić dowolny podzbiór wartości w serii podziałów).
- Braki danych wymagają imputacji dla `DecisionTreeClassifier` (mediana dla numerycznych, moda dla katagorycznych).
`HistGradientBoostingClassifier` obsługuje NaN natywnie.
- Dla **klasyfikatora k -NN** odwrotnie: skalowanie jest absolutnie konieczne, w przeciwnym razie cecha o największej skali zdominuje metrykę.

6. Plan ćwiczenia: drzewa decyzyjne

1. Załaduj zbiór *Adult Census Income* przez `fetch_openml('adult', version=2)` i wykonaj eksploracyjną analizę danych: typy kolumn, braki danych, rozkład klas, podział cech na numeryczne i katégoryczne, $P(\text{target}=1 \mid \text{cecha})$ dla cech katégorycznych.
2. Zbuduj pipeline preprocessingu używając `ColumnTransformer`, łącząc `SimpleImputer` z `OrdinalEncoder` dla katégorii. Dokonaj stratyfikowanego podziału 75%/25%.
3. Naucz *pełne drzewo* (bez regularyzacji) i zaobserwuj overfit: różnicę między dokładnością na zbiorze treningowym i testowym. Jaka głębokość drzewa została osiągnięta?
4. Za pomocą 10-krotnej walidacji krzyżowej dla $\text{max_depth} \in \{1, \dots, 20\}$ zilustruj *krzywą bias-variance*. Zidentyfikuj optymalną głębokość.
5. **Cost-Complexity Pruning**. Dla pełnego drzewa wywołaj: `full_tree.cost_complexity_pruning_path()` uzyskując sekwencję $\alpha_0 < \alpha_1 < \dots < \alpha_m$. Dla każdego α :
 - wytrenuj drzewo z `ccp_alpha= $\alpha$` ,
 - zapisz głębokość, liczbę liści, train accuracy, test accuracy,
 - zwizualizuj train/test accuracy oraz liczbę liści w funkcji α (patrz Rysunek 1).



Rysunek 1: Oczekiwany kształt krzywej CCP. Przy $\alpha \rightarrow 0$ drzewo jest pełne (overfit); przy $\alpha \rightarrow \alpha_{\max}$ drzewo redukuje się do korzenia. Optimum wyznaczamy przez CV.

6. Wyznacz optymalne α^* przez `GridSearchCV` z `param_grid={'ccp_alpha': alphas}` na 10-krotnej CV.
7. **Porównanie strategii regularyzacji**. Zbuduj tabelę porównawczą czterech modeli:
 - a) pełne drzewo,
 - b) pre-pruning z `max_depth=best_d`,
 - c) pre-pruning kombinowane (`max_depth=10`, `min_samples_leaf=20`, `min_impurity_decrease=1e-4`),
 - d) post-pruning z `ccp_alpha= $\alpha^*$` .
 Kolumny: głębokość, liczba liści, train/test accuracy, f1, overfit gap. *Skoomentuj*: którą strategię wybrałbyś do produkcji i dlaczego?
8. **Wizualizacja drzewa**. Dla drzewa ograniczonego do głębokości 3 wygeneruj:

- wizualizację przez `export_graphviz()` → `graphviz.Source()`,
- wizualizację przez `dtreeviz` (histogramy rozkładu w węzłach, pie charts w liściach).

Porównaj obydwie reprezentacje.

9. **Interpretowalność.** Dla drzewa przyciętego CCP wyznacz trzy rodzaje *feature importance*:

- a) built-in (`feature_importances_`),
- b) `permutation_importance` na zbiorze testowym,
- c) wartości SHAP (`shap.TreeExplainer`): `shap.summary_plot` dla 1000 losowych rekordów oraz `shap.plots.waterfall` dla jednej wybranej predykcji.

Skomentuj: czy rankingi z trzech metod są spójne? Skąd mogą się brać różnice?

10. * Wykonaj *fairness audit*: czy model przewiduje klasę pozytywną z różną dokładnością dla podgrup ze względu na płeć (`sex`) i rasę (`race`)? Porównaj *demographic parity* i *equalized odds* w obu podgrupach.

7. Plan ćwiczenia: lasy losowe

Przed implementacją, zapoznaj się z kluczowymi koncepcjami teoretycznymi:

1. **Dekompozycja błędu**: $\mathbb{E}[(\hat{f}(x) - y)^2] = \text{bias}^2 + \text{variance} + \sigma_{\text{noise}}^2$. Drzewo głębokie ma niski bias i wysoką wariancję.
 2. **Bagging** (Breiman 1996): B niezależnych próbek bootstrap generuje B drzew. Wariancja ich średniej: $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, gdzie ρ - uśredniona korelacja między drzewami.
 3. **Random Forest** (Breiman 2001): dodatkowa randomizacja przez `max_features` redukuje ρ , znacząco poprawiając efektywność bagging-u.
 4. **Out-of-bag**: każde drzewo jest uczone na $\approx 63\%$ unikalnych próbek; pozostałe $\approx 37\%$ (OOB) służą jako darmowy estymator generalizacji.
1. Zbadaj zbieżność lasu losowego przy wzroście `n_estimators`. Wykorzystaj `warm_start=True`: dla $n \in \{10, 20, 50, 100, 200, 300, 500\}$ dotrenuj model i zapisz OOB score oraz test accuracy. Narysuj krzywą zbieżności.
 2. Zbadaj wpływ `max_features` $\in \{1, 2, 4, \text{'sqrt'}, \text{'log2'}, 0.5, \text{None}\}$ przy `n_estimators=200`. Zbuduj tabelę OOB vs test. *Skomentuj*: jaka wartość daje najlepszy kompromis? Dlaczego skrajne wartości szkodzą?
 3. * **Optymalizacja bayesowska**. Korzystając z biblioteki Optuna zoptymalizuj hiperparametry RF:

```
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 500, step=50),
        'max_depth': trial.suggest_int('max_depth', 3, 30),
        'max_features': trial.suggest_categorical('max_features',
                                                  ['sqrt', 'log2', 0.3, 0.5, 0.8]),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 30),
    }
    rf = RandomForestClassifier(**params, random_state=42, n_jobs=-1)
    return cross_val_score(rf, X_train, y_train, cv=5,
```

```

        scoring='balanced_accuracy').mean()
study = optuna.create_study(direction='maximize',
                           sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=25)

```

4. Porównaj najlepszy Random Forest z `HistGradientBoostingClassifier` (domyślne parametry, `max_iter=300`). Zbuduj tabelę: Decision Tree (CCP) / Random Forest (Optuna) / HistGradientBoosting. *Skomentuj*: który model wygrywa i jaką cenę płacisz za poprawę jakości?
5. * Powtórz analizę z Rozdziału 6 (interpretowalność) dla Random Forest. Jak zmieniają się rankingi cech w stosunku do pojedynczego drzewa?

8. Plan ćwiczenia: klasyfikator k -NN

1. Załaduj zbiór *Palmer Penguins* (`fetch_openml('penguins', version=1)`). Usuń NaN, zakoduj cechy kategoryczne i etykietę, **zastosuj** `StandardScaler`. *Uzasadnij* konieczność skalowania dla k -NN.
2. Przetestuj kombinacje $k \in \{1, 3, 5, 7, 9, 11, 15\}$, `metric` $\in \{\text{euclidean, manhattan, chebyshev}\}$, `weights` $\in \{\text{uniform, distance}\}$. Dla każdej kombinacji: 5-krotna CV accuracy. Zwizualizuj wyniki heatmapą.
3. Powtórz analizę dla zbioru NYT-frame (zredukowanego przez PCA do 10 wymiarów). Zaobserwuj, jak wybór metryki zależy od wymiarowości.
4. * Implementacja własna KD-tree. Przetestuj na 10 000 losowych punktów 10D porównując czas wyszukiwania NN: brute-force (NumPy) vs `sklearn.neighbors.KDTree` vs `scipy.spatial.cKDTree`.
5. * Wykonaj optymalizację bayesowską hiperparametrów k -NN (`scikit-optimize` lub Optuna) w przestrzeni:

```

param_space = {
    'n_neighbors': (1, 20),
    'metric': ['euclidean', 'manhattan', 'cosine'],
    'weights': ['uniform', 'distance']
}

```

9. Raport

Końcowym deliverable laboratorium jest:

- Jupyter notebook (.ipynb) z zakomentowanym kodem i wynikami,
- Komentarze Markdown w Jupyter notebook odpowiadające na pytania:
 1. Która strategia regularyzacji drzewa (pre-pruning, post-pruning, hybryda) okazała się najskuteczniejsza i dlaczego?
 2. Czy rankingi cech według built-in importance, permutation importance i SHAP są spójne? Jak interpretować rozbieżności?
 3. Jaki jest zysk Random Forest w porównaniu z pojedynczym drzewem, oraz gradient boostingu w stosunku do RF? Jaka jest cena tego zysku?
 4. Jak zachowuje się trade-off bias-variance w Random Forest w funkcji `max_features`?

5. Jakie zagrożenia fairness wiążą się ze stosowaniem modelu trenowanego na zbiorze Adult Census?

10. Sposób oceny

- Zadania podstawowe (sekcje 6.1–6.8, 7.1–7.4, 8.1–8.3): 65% punktów.
- Zadania oznaczone gwiazdką (*): do 35% dodatkowych punktów.
- Jakość raportu i komentarzy interpretujących wyniki: modyfikator $\times 0,8$ – $\times 1,2$.

11. Literatura

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (wyd. 2). Springer. Rozdz. 9 (drzewa), 15 (Random Forests), 10 (boosting).
- Breiman, L. (2001). Random Forests. *Machine Learning* 45: 5–32.
- Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. A. (1984). *Classification and Regression Trees*. CRC Press. [oryginalne źródło CCP]
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence* 2: 56–67.
- Parr, T., & Grover, P. (2020). dtreeviz: a decision tree visualization library. <https://github.com/parrt/dtreeviz>.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD*.
- Aggarwal, C. C. (2015). *Data Mining: The Textbook*. Springer.
- Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (wyd. 3). Elsevier.